

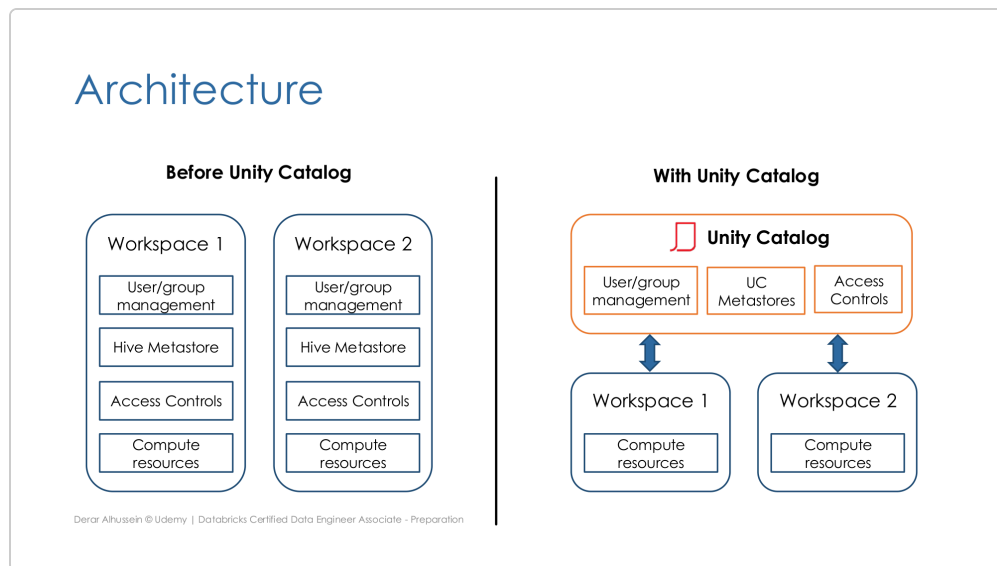
Unity Catalog vertieft: Architektur, Identitäten und vollständiges Rechemodell

Section 7 · Governance and Security · Quelle: Udemy-Kursmaterial „Unity Catalog“ & „Data object privileges“, ergänzt mit Databricks-Dokumentation

Kapitel 1 dieser Section hat das Unity-Catalog-Sicherheitsmodell (ACLs, Tagging, Lineage, System-Tables) vorgestellt, Kapitel 7 die Befehle `GRANT` / `REVOKE` / `DENY` vertieft. Dieses Kapitel ergänzt drei bislang fehlende Bausteine: die **Architektur**-Perspektive (warum Unity Catalog eingeführt wurde), das **Identitätsmodell** (Users, Service Principals, Groups, Identity Federation), sowie das **vollständige Rechemodell** im Vergleich zum älteren, Hive-Metastore-basierten Berechtigungssystem.

1. Architektur: Governance vor und nach Unity Catalog

Vor Unity Catalog verwaltete jeder Databricks-Workspace seine Nutzerverwaltung, seinen Hive Metastore und seine Zugriffskontrollen vollständig **unabhängig** – Rechte mussten in jedem Workspace einzeln gepflegt werden, auch wenn dieselben Personen und dieselben Daten betroffen waren. Unity Catalog verlagert Nutzerverwaltung, Metastores und Zugriffskontrollen auf die **Account-Ebene** (verwaltet über die Account Console) und macht sie dadurch workspace-übergreifend wiederverwendbar: Ein einziger Unity-Catalog-Metastore lässt sich mehreren Workspaces gleichzeitig zuweisen, sodass diese sich Zugriffskontrollen und Datenzugriff teilen, statt sie zu duplizieren.



Vor Unity Catalog verwaltete jeder Workspace Nutzer, Metastore und Zugriffskontrollen unabhängig; Unity Catalog zentralisiert alle drei auf Account-Ebene und teilt sie über Workspaces hinweg.

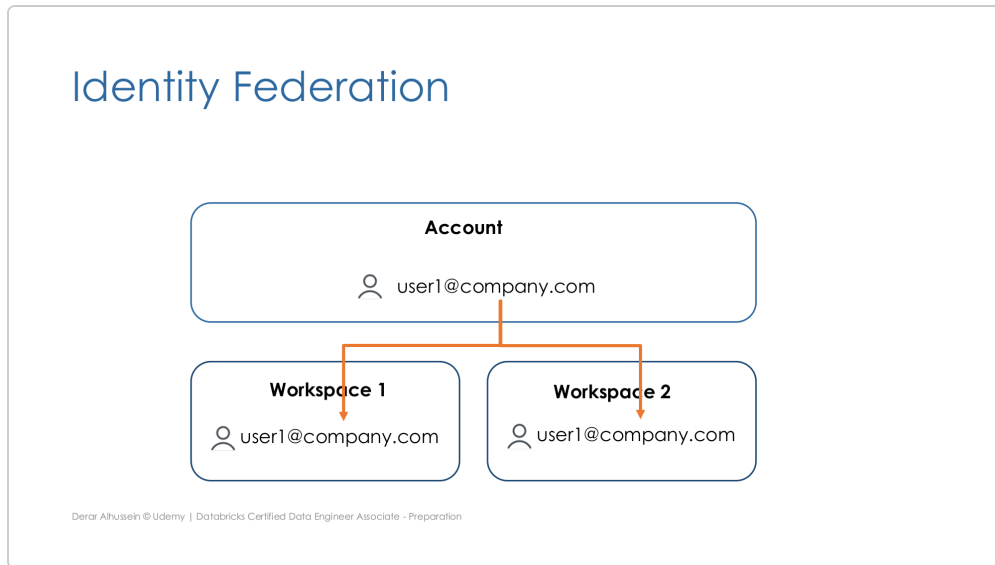
2. Identitäten: Users, Service Principals, Groups

Unity Catalog unterscheidet drei Arten von Prinzipalen (siehe auch die **GRANT** -Beispiele in Kapitel 7):

- **User** – eine natürliche Person, eindeutig identifiziert über ihre E-Mail-Adresse. Ein User kann zusätzlich die Rolle **Account Administrator** erhalten, um Metastores zu verwalten und Workspaces zuzuweisen.
- **Service Principal** – eine Identität für automatisierte Tools und Anwendungen (z. B. eine CI/CD-Pipeline), eindeutig identifiziert über eine Application ID statt einer E-Mail-Adresse. Auch Service Principals können administrative Rechte erhalten, um programmatisch Verwaltungsaufgaben auszuführen.
- **Group** – fasst Users und Service Principals zu einer Einheit zusammen und kann selbst weitere Gruppen enthalten (verschachtelte Gruppen, z. B. eine übergeordnete Gruppe „employees“ mit den Untergruppen „HR“ und „Finance“).

Identitäten existieren dabei auf zwei Ebenen: auf **Account-Ebene** (zentral in der Account Console verwaltet) und auf **Workspace-Ebene**. **Identity Federation** erlaubt es, eine Identität einmalig auf Account-Ebene anzulegen und sie anschließend beliebigen Workspaces zuzuweisen – ohne dass in jedem Workspace eine eigene Kopie derselben Identität manuell gepflegt werden müsste.

Identity Federation



Identity Federation: Eine einmal auf Account-Ebene angelegte Identität lässt sich mehreren Workspaces zuweisen, ohne sie dort erneut anzulegen.

3. Weitere Securable Objects: Storage Credential, External Location, Share, Recipient

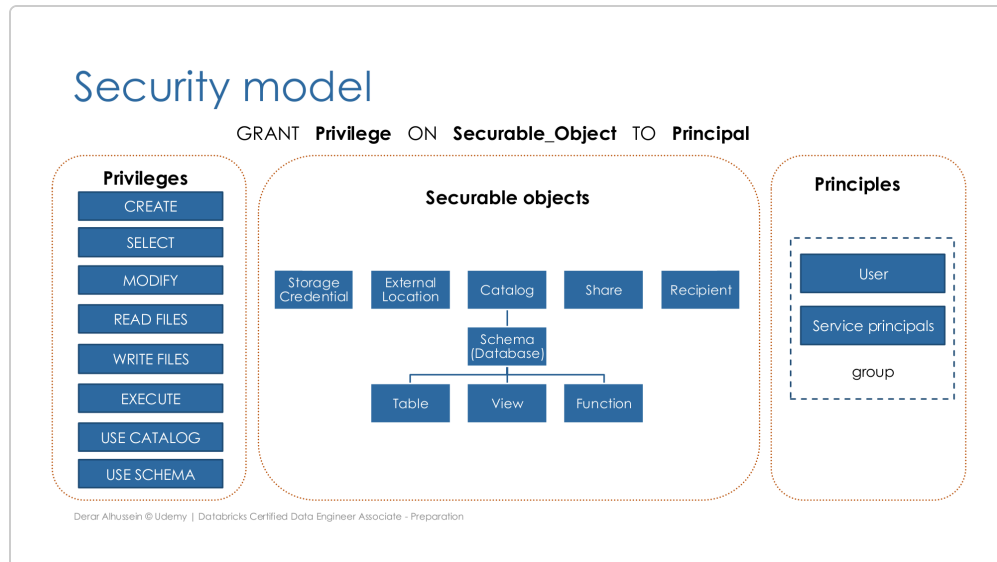
Neben der bereits bekannten Objekthierarchie (Metastore → Catalog → Schema → Table/View/Function) verwaltet Unity Catalog zwei weitere Objektkategorien: **Storage Credentials** kapseln die Authentifizierung gegenüber dem zugrunde liegenden Cloud-Speicher und gelten für einen gesamten Speicher-Container; **External Locations** referenzieren darauf aufbauend ein konkretes Verzeichnis innerhalb dieses Containers. Für Delta Sharing kommen zusätzlich **Shares** (eine Sammlung von Tabellen, die mit einem oder mehreren **Recipients** geteilt wird) hinzu.

4. Das vollständige Privilegien-Modell

Kapitel 7 hat `GRANT` / `REVOKE` / `DENY` anhand einzelner Beispiele (`SELECT` , `USE SCHEMA` , `ALL PRIVILEGES`) gezeigt. Die vollständige Liste der in Unity Catalog verfügbaren Privilegien:

Privileg	Bedeutung
<code>CREATE</code>	Ein neues Objekt anlegen (z. B. eine Tabelle in einem Schema)
<code>SELECT</code>	Lesezugriff auf ein Objekt
<code>MODIFY</code>	Daten in einem Objekt hinzufügen, ändern oder löschen

READ FILES	Zugriff auf die zugrunde liegenden Dateien eines Storage-Objekts (Volume, External Location)
WRITE FILES	Schreibzugriff auf die zugrunde liegenden Dateien eines Storage-Objekts
EXECUTE	Eine benutzerdefinierte Funktion (Function) ausführen
USE CATALOG / USE SCHEMA	Voraussetzung, um überhaupt auf Objekte innerhalb des Catalogs bzw. Schemas zugreifen zu können (siehe Kapitel 7)



Das Sicherheitsmodell von Unity Catalog verbindet drei Bausteine: Prinzipale (wer), Securable Objects (was) und Privilegien (wie) – vergeben über `GRANT Privilege ON Securable_Object TO Principal`.

5. Legacy: weiterhin Zugriff auf den Hive Metastore

Unity Catalog ist **additiv**: Der bisherige, workspace-lokale Hive Metastore bleibt nach Aktivierung von Unity Catalog vollständig zugänglich – unabhängig davon, welcher Unity-Catalog-Metastore dem Workspace zugewiesen ist, stellt der feste Catalog-Name `hive_metastore` weiterhin Zugriff auf die dort verwalteten Datenbanken und Tabellen bereit. Eine harte, erzwungene Migration ist damit nicht nötig – bestehende Hive-Metastore-Objekte und neue Unity-Catalog-Kataloge (z. B. `dev`, `prod`) lassen sich parallel referenzieren.

Ergänzt aus der offiziellen Databricks-Dokumentation: Vor Unity Catalog nutzte Databricks für Governance ein älteres, Hive-Metastore-basiertes **Table-ACL-Modell** mit einem eigenen, leicht abweichenden Satz an Privilegien: `SELECT`, `MODIFY`, `CREATE`, `READ_METADATA` (Metadaten eines Objekts einsehen), `USAGE` (ohne inhaltliche Wirkung, aber Voraussetzung für jede Aktion auf einem Datenbankobjekt) und `ALL PRIVILEGES`, vergeben ebenfalls über `GRANT ... ON ... TO ...` auf

die Objekttypen CATALOG , SCHEMA , TABLE , VIEW , FUNCTION sowie ANY FILE für Zugriff auf das zugrunde liegende Dateisystem. Wer Rechte vergeben durfte, richtete sich nach der Objekt-Eigentümerschaft (Databricks-Administrator: alle Objekte im Catalog und Dateisystem; Catalog-Owner: alle Objekte im Catalog; Database-Owner: alle Objekte in der Datenbank; Table-Owner: nur die eigene Tabelle). Unity Catalog übernimmt das Grundprinzip (GRANT / REVOKE , Owner-basierte Rechtevergabe), ersetzt aber ANY FILE durch die feingranularen READ FILES / WRITE FILES - Privilegien auf Volumes bzw. External Locations und führt zusätzlich USE CATALOG als neue, durch die dritte Namensebene (Catalog) nötig gewordene Voraussetzung ein – USAGE und READ_METADATA entfallen dagegen als eigenständige Privilegien.

Quelle: [What is Unity Catalog? – Databricks-Dokumentation](#) · [Manage users and groups – Databricks-Dokumentation](#)